FlashAttention 与 MixCV

1.**官方 expert 版** `examples/flash_attn_npuir.py` **已通过 JIT 兼容层跑通**：Expert codegen 生成 `_mix_aic/_mix_aiv` 后，把旧属性 `syn_instr_mode` 改写为 CANN 9.0.0 接受的 `sync_instr_mode`，并对后端偶发 SIGSEGV 自动重试；seq_len=512/1024/4096 均 `All check passed!`。**Developer 版也跑通**，seq_len=512/1024/2048/4096
2.四个形状 `assert_close(rtol=atol=1e-2)` 全过，max_abs ≤ 3.7e-4。**kernel 设备时长**（CANN `op_summary`）27.3 / 56.6 / 208.6 / 824.6 μs，对 torch 三段式（softmax(QK^T)V）是 **0.62x / 1.42x / 2.22x / 3.76x**——小尺寸融合赢，大尺寸下当前实现吞吐不足；64×64 小 tile 是首要假设，尚未由 sweep 证实。端到端 0.42 / 0.45 /0.63 / 1.28 ms，对 torch 是 1.40x / 1.45x / 1.91x / 3.60x（512 那一档主要受 host 地板影响）。
3.机制上记住四句话：**Cube 算 GEMM、Vector 算 softmax；expert 模式按** `T.Scope` **显式拆 AIC/AIV，Developer 模式发一个 MIX 函数交给 HIVM 自动拆；910B3 是 20:40（1:2），每个 Cube 块配两个 AIV子块用** `subid` **分行；代码请求** `T.Pipelined(num_stages=2)` **+ HIVM auto multi-buffer，按设计用于 Cube/Vector 重叠，但本轮还没用 A/B 或后端 IR 确认实际生效。**

0. 测量口径

列	定义	数据来源
`host_us`	不 sync 连打 20 次，总时间 / 20	`time.perf_counter()`
`event_ms`	每次调用前后各记一个 NPU Event，10 轮中位数	`torch.npu.Event.elapsed_time`
`e2e_ms`	每次调用后 `torch.npu.synchronize()`，10 轮中位数	`time.perf_counter()`
`device_us`	kernel 自身设备时长	CANN `op_summary_*.csv` 的 `Task Duration(us)`

- warmup 3 次丢弃；报中位数 + IQR。
- **event 列不是纯设备时间**：start event 入队后，host 还要花时间把 kernel 入队；只要队列是空的，设备会在 start event 之后空等这段 host 时间。本轮实测 `event_ms` 比 `e2e_ms` 稳定高约 0.05–0.07 ms（512：0.486 vs 0.420），与 event 记录本身的开销同量级。**绝对设备时间以** `op_summary` **为准**；对拍比值用同一口径的 `event` 或 `e2e` 都可以，但不能拿 event 当 kernel 时间算 TFLOPs。
- host 列在大 kernel 上会退化成队列反压（Day 4 的 4096^3 已记录）：seq_len=4096 时host 1211 μs ≈ kernel 时间，不是真实提交成本。

1. 运行结果

1.1 官方 expert 例子：初次失败，兼容修复后跑通

命令与日志：`python examples/flash_attn_npuir.py` → `logs/run_default.log`

```
error: Dialect `' not found for custom op 'syn_instr_mode'
```

IR 里出现 4 处（`logs/fa_expert_ir.npuir`）：

```
hivm.hir.sync_block_set[<CUBE>, <PIPE_FIX>, <PIPE_S>] flag = %9 syn_instr_mode =
<INTRA_BLOCK_SYNCHRONIZATION>
hivm.hir.sync_block_set[<VECTOR>, <PIPE_MTE3>, <PIPE_S>] flag = %26 syn_instr_mode =
<INTRA_BLOCK_SYNCHRONIZATION>
```

三次定位尝试：

#	做法	结果	日志
1	CANN 9.0.0 自带 `bishengir-compile`	解析 IR 就失败（属性不存在）	`run_default.log`
2	仓库 3rdparty 构建的 `bishengir-compile`（llvm 19.1.7，与 IR 匹配）	解析过； 调 hivmc 时 `--enable-cpu-trace-intrinsic` / `--enable-lir-compile` 未知	`run_expert_repo_compiler.log`
3	wrapper 删属性 / 过滤 hivmc flag / 连 hivmc 输入一起 strip	删属性可出 `.o`，但后续 pass crash；过滤 flag 后 `nd2nz` legalize 失败	`run_default_wrapped.log`、`run_`

根因【源码+日志】：expert codegen 在 `src/target/codegen_npuir.cc:1026` 硬编码打印 `syn_instr_mode`；生成 IR 已有 `_mix_aic/_mix_aiv`，说明 Expert 模式确实开启并完成拆分。CANN 9.0.0 的 `bishengir-compile` 使用新属性名 `sync_instr_mode`；完成改名后，`ConvertHFusionToHIVM` 在 MIX 模块上还会非确定性 SIGSEGV，同一输入重试可通过。修复落在 `tilelang/jit/jit_npu.py`：按编译器版本改写属性，并对信号退出做最多 8 次重试；repo 自带 19.1.7 编译器路径仍因缺少配套 hivmc 而不可用，但已不再需要。

修复后验证（`rtol=atol=1e-2`）：

seq_len	结果	日志
512	`All check passed!`	`logs/expert_seq512_fixed.log`
1024	`All check passed!`	`logs/expert_seq1024_fixed.log`
4096	`All check passed!`	`logs/expert_seq4096_fixed.log`

三份日志都记录了首次编译被 signal 11 杀死、随后重试成功。这里只证明 expert 正确性链路已通；日志没有设备时长，不能拿 Developer 性能数字代表 Expert。

1.2 Developer 例子：跑通

命令与日志：`python examples/flash_attn_npuir_dev.py` → `logs/run_dev_example.log`

信号	结果
`AscendNPU IR compile success`	☑
`Precompiling NPU launcher header...`	☑
`device='npu:0'`（无 CPU fallback）	☑
`All check passed.`	☑

形状：seq_len=512，dim=128，block_M/N/K=64/64/32，dtype=fp16，accum=fp32。

1.3 形状与耗时（Developer 模式，10 轮）

`logs/bench_fa_dev.log`、`bench_fa_dev.csv`；正确性均为 `assert_close(rtol=atol=1e-2)`。

seq_len	block	host μs/call	event ms [IQR]	e2e ms [IQR]	torch event ms	torch e2e ms	比值 (e2e)	TFLOPs†	max_abs
512	64/64/32	358.9	0.486 [0.022]	0.420 [0.029]	0.345	0.301	1.40x	0.32	3.66e-4
1024	64/64/32	388.3	0.500 [0.022]	0.452 [0.018]	0.354	0.311	1.45x	1.22	2.44e-4
2048	64/64/32	562.6	0.678 [0.018]	0.631 [0.017]	0.379	0.330	1.91x	3.46	1.41e-4
4096	64/64/32	1211.5	1.329 [0.033]	1.275 [0.034]	0.378	0.354	3.60x	6.76	3.66e-4

† TFLOPs 用 e2e 时间算（$4 \cdot seq^2 \cdot dim$，QK+PV 各 $2 \cdot seq^2 \cdot dim$），**含 host 与同步开销**，是端到端等效值，不是 kernel 峰值；kernel-only 数字见 §1.4。

读法：

- 小尺寸（512/1024）e2e 被 host 地板压住：host 359/388 μs 与 e2e 420/452 μs 只差 60 μs，与 Day 3 的 216–232 μs host 地板属于同一类开销（这里 block 少、launcher 底价占主导）。
- 2048 开始设备时间抬头；4096 的 host 列 1211 μs 已经是队列反压（≈kernel），**不能当提交成本。**
- torch 三段式需要在独立算子之间物化完整 `seq×seq` 中间张量，我们的融合 kernel 不在 GM 中物化完整 S/P；但当前实现仍慢 3.6x。**64×64 tile 是首要排查项，不是已由实验确认的唯一根因。**
- 比值随 seq_len 变大而变差：512 的 1.40x 里 host 占大头（设备侧我们反而快 1.6x）；4096 的 3.60x 才是 kernel 差距（设备侧 3.76x）。

1.4 设备时间（CANN `op_summary`）

见 `logs/profile_fa_dev.log`、`logs/profile_fa_dev_1024_2048.log`（实验包内已扁平化为 `logs/profiler/op_summary/`，文件名与原 run 目录的对应关系见 `manifest.csv`；仓库里仍是 `logs/profiler/export_only_prof_dir/`）。

Developer 模式的 MIX kernel 在 profiler 里是一条 `flash_attention` 记录（不是 `_mix_aic`/`_mix_aiv` 两条；那两条只在 expert 模式显式拆函数时出现），所以 `Task Duration` 就是整个 kernel 的墙钟时间，可以直接用。

seq_len	ours 设备 μs	torch 三段式设备 μs†	设备比值	ours TFLOPs	torch TFLOPs
512	**27.33**	44.34	**0.62x**（我们快 1.6x）	4.91	3.03
1024	**56.60**	39.97	1.42x	9.49	13.43
2048	**208.61**	93.91	2.22x	10.29	22.87
4096	**824.57**	219.48	3.76x	10.42	39.14

† torch 三段式按同一 session 内各 op 的中位数相加：512 = 2×4.49(matmul) + 2×7.21(cast) + 5.43(mul) + 15.51(softmax)；
1024 = 2×7.84 + 2×3.25 + 6.59 + 11.20；2048 = 2×18.42 + 2×7.64 + 10.78 + 31.01；
4096 = 2×28.31 + 2×18.86 + 27.56 + 97.58。torch 这些 op 在 stream 上是串行的，相加成立。

读法：

- **ours 的 TFLOPs 从 1024 起稳定在 ~10**（9.49 / 10.29 / 10.42），说明当前实现已进入吞吐平台；64×64 tile 是优先验证的假设，但同步、Vector 规约和多缓冲是否生效也尚未排除。
- **torch 从 3.03 一路涨到 39.14 TFLOPs**：它的 matmul 是库调优的大 tile，但代价是 materialize `seq×seq` 中间矩阵（512 时 0.5 MiB，4096 时 32 MiB）。
- **交叉点在 512–1024 之间**：512 时融合版本设备侧更快，1024 起 torch 基线反超。下一步先做合法 tile sweep；当前 Q load 没有 M 尾块保护，所以 seq=4096 时先测能整除的 block_M=64/128，block_M=96 要补 tail 后再测。
- e2e 表（§1.3）在 512 那行显示 1.40x，**这不是 kernel 慢**：kernel 27 μs，host+event 约 450 μs。任何小于 ~0.5 ms 的 kernel 都会被 host 地板淹没，必须看 `op_summary` 列。

2. mixcv skill 摘要

skill 在 `references/mixcv-skill/`：4 份 md、合计 3.5 KB。**它不是教程，是一张路由表 + 一份最小模式清单**——只回答"什么样的 kernel 算 MixCV""该抄哪个模板""四个原语叫什么"，具体语义、边界和坑要自己回源码 / 官方文档补。

2.1 `SKILL.md`：触发条件与三条强制规则

frontmatter 里 `name: tilelang-mixcv-skill`，`description` 列出的触发词是：flash attention、mixcv、online softmax、流水并行、`sync_block_set/wait`、`Scope("Cube")+Scope("Vector")`、`PIPE_FIX`、跨核 workspace 协同、融合算子性能调优。

三条 **Mandatory** 规则：

规则	原文含义	对我们的影响
Docs Auto Routing	先按 `AGENTS.md` 的 "Docs Auto Routing Rules" 决定读哪些文档	skill 不复制文档，只做索引；细节顺着索引读
Operator baseline	写新 MixCV 算子前，先翻 `examples/` 和 `testing/npuir/`，**优先改现成算子而不是从零写**	FA 直接改 `flash_attn_npuir_dev.py`，没有重写
Developer 模式识别	同一 kernel 里，Cube 侧有 `T.gemm` **且** Vector 侧有任意一个 v 前缀算子（`vmul/vadd/vexp/vcast/vbrc`…）→ 按 MixCV 处理，**用户没说 mixcv 也要触发**	dev 版 FA 正好命中：2 个 `T.gemm` + 一片 `vexp/vmul/vcast`

第三条是这份 skill 最有价值的地方：它把 "MixCV" 从一个**写法规约**（有没有 `T.Scope`）变成了一个**可判定的结构特征**（同一个 `prim_func` 里同时出现矩阵乘和逐元素算子）。所以 dev 版 FA 虽然一行 `T.Scope` 都没有、在 TileLang 层看只是"普通 kernel"，读的时候仍应按四步生产者-消费者模型去理解，而不是按单个 kernel 读。

`Key primitives` 只列了 4 个：`T.Scope("Cube")/("Vector")`、`T.rs("PIPE_FIX")` 等 pipe region、`T.sync_block_set` / `T.sync_block_wait`、`T.Pipelined`。
`Official docs to consult` 指了 5 份官方文档（我确认仓库里都存在）：`同步管道操作/T.sync_block_set.md`、`T.sync_block_wait.md`、`T.pipe_barrier.md`、`线性代数操作/T.gemm.md`、`数学操作/T.vexp.md`。`Related skills` 指向 `tilelang-cube-skill` / `tilelang-vector-skill` / `tilelang-debug-helper`——即 MixCV ≈ cube skill + vector skill 的合成，外加两者之间的同步协议。

2.2 `references/pipeline.md`：四步交接 + 三条 good practice

四步：**Cube 写 workspace** → `sync_block_set` → **Vector** `sync_block_wait` → **Vector 消费。**
三条 good practice 各对应一个真实的坑：

原文	展开	在我们 kernel 里的落点
keep sync id consistent between set and wait	set / wait 的 id 必须一一对应；id 是**逻辑迭代号**，不是计数器	expert FA 里 Cube `sync_block_set(i)` 通知第 i 块 S 就绪，Vector `sync_block_wait(i)` 等它
minimize workspace footprint by tiling	workspace 在 GM（HBM）上，能按 tile 分片就别整块 materialize	我们只落 `S` 和 `P`，不落全量 `O`；4096 时各 32 MiB
isolate stage-specific memory buffers	每个 stage 用自己的 buffer / L1 slot，不要复用同一个 L1 让两条流水互相踩	expert 版 QK 的 `l1_a/l1_b` 与 P/V 的 L1 是分开的

注意它只描述了单向（Cube→Vector）。FA 实际是两轮：PV 阶段要把 `P` 从 Vector 回传给 Cube，flag 需要两组、方向相反（详见 §3.1）——这一层 skill 里没有。

2.3 `references/flash-attn-pattern.md`：A/B/C 三段与 expert FA 的对应

skill 的 stage	做什么	expert 代码位置
A：Cube 算 score	tile Q/K → gemm → 写 workspace → `sync_block_set`	`flash_attn_npuir.py` L75-119（`T.gemm(b_transpose=True)` + fixpipe → `workspace_1`）
B：Vector 做 softmax	wait → cast/scale/exp/reduce → 维护 running max/sum	L163-241（`vcast→vmul(scale)→reduce(max)→vmax→vsub→vexp→vcast(fp16)` 写 `workspace_2`，再 `reduce(sum)` 更新 `logsum`）
C：PV + 输出	消费 V tile → 累加 → 归一化 → 写回	Cube 部分 L121-158；Vector 部分 L243-268（`acc_o*scale + PV`，最后 `vdiv(logsum)`）

skill 把 Stage C 写成"Vector 一段"，真实实现里 **C 又拆成 Cube 的 PV 和 Vector 的累加两半。**读 skill 只拿得到骨架，哪半落在哪个核必须回代码确认。

2.4 `references/flash-attn-dev.md`：Developer 版差异与迁移建议

三句话：① 再定义一遍 MixCV 判据（同上）；② 典型写法是 `alloc_shared` / `alloc_fragment` + v 前缀 + `T.Pipelined`；
③ **迁移建议：先在 Developer 风格把正确性跑通，再把热点路径迁到 Expert 风格。**

我们这周就是按这个顺序走的：dev 版四个形状全过（§1.2 / §1.4）；expert codegen 已拆出 AIC/AIV，补上属性名兼容和 SIGSEGV 重试后，512/1024/4096 也已跑通（§1.1）。当前剩余的是 expert 性能测量和长线工具链对齐，不再是正确性阻塞。

2.5 skill 没写、这次自己补上的四件事

1. **1:2 配比与 `subid` 分行**：skill 只字未提 20 Cube / 40 Vector，也没说 `T.Kernel` 第二个返回值 `vid` 的范围是块数×2。这部分是从 `Ascend910B3.ini:17-33` 和 `src/ir.cc:178-190` 补的（§3.3）。
2. **双缓冲是两层机制**：skill 只有一句 “Pipelined loops where suitable”。代码侧 `T.Pipelined(num_stages=2)` 管循环分阶段，HIVM `--enable-auto-multi-buffer` 请求 buffer 多槽；文档配置的 GM workspace 默认值为 2，但本轮尚未确认后端最终采用了两槽（§3.4）。
3. `T.rs` **不一定触发 MIX**：源码注释写 “iff exists T.rs”，但 `T.rs("PIPE_FIX")` 设的 attr key 是 `"PIPE_FIX"` 而不是 `"resource_scope"`；真正触发 expert 拆分的是 `T.Scope("Vector")`（`src/ir.cc:366-372`）。**以 IR 为准**，不要照注释推（§3.2）。
4. **两种模式的 IR 不同构**：dev 只发一个 `func_core_type=MIX` 函数，拆核由下游 CV Pipelining pass 完成；expert 在 codegen 阶段就拆成 `_mix_aic` / `_mix_aiv`。skill 同时覆盖两种模式，却没提醒“产物结构不同、不能跨模式比较 IR”。

2.6 仓库规则（`AGENTS.md`，skill 第一条 Mandatory 指向它）

优先从 `examples/` 和 `testing/npuir/` 找最近模板，不要凭空新写；API 默认用 `T.vmul` 这类 v 前缀，`npuir_xxx` 只做兼容读。

3. 机制

3.1 Cube / Vector 分工

以 expert 版 `examples/flash_attn_npuir.py` 为例：

阶段	位置	做什么
QK^T	Cube，L75-119	Q/K tile 搬 L1（`l1_a`、`l1_b`），`T.gemm(..., b_transpose=True)` 累加到 L0C，`PIPE_FIX` 里 fixpipe 到 `workspace_1`，然后 `sync_block_set(i)`
online softmax	Vector，L163-241	`sync_block_wait(i)` 等 S tile；`vcast`、`vmul(scale)`、`reduce(max)`、`vmax` 维护 running max、`vsub` + `vexp`、`vcast` 成 fp16、写 `workspace_2`（P）、`reduce(sum)`、更新 `logsum`
PV	Cube，L121-158	从 `workspace_2` 搬 P、从 V 搬 tile，`T.gemm` 到 L0C，fixpipe 到 `workspace_3`（按 i 分片，不覆盖）
输出累加	Vector，L243-268	读 `workspace_3`，`vmul(acc_o, scale)`、`vadd` 累加，最后 `vdiv(logsum)`、`vcast`、写 `Output`

一句话：**Cube 只做矩阵乘（QK^T、PV），Vector 只做逐元素/规约（softmax、累加、归一化）**；交接全部通过 GM workspace + `sync_block_set/wait` 的 flag。`T.rs("PIPE_FIX")` / `T.rs("PIPE_MTE2")` / `T.rs("PIPE_MTE3")` 把同步点钉到具体硬件流水上（fixpipe、搬入、搬出），避免编译器把 flag 插到错误的 pipe。

3.2 CV 自动拆分

有两层“拆”：

1. **TileLang expert 模式：显式拆成两个函数。** `T.Scope("Cube")` / `T.Scope("Vector")` 在 TIR 里变成 `resource_scope` 属（`src/ir.cc:405-416`，Cube=0、Vector=1）。codegen 先推断 core type：

出现 `resource_scope=1` 或 `T.rs` → MIX（`codegen_npuir_dev.cc:265-281`）；出现 `tl.npuir_dot` → AIC（`codegen_npuir.cc:1661-1668`）。然后 MIX 会为 AIC/AIV 各生成一个函数，函数名加后缀 `_mix_aic` / `_mix_aiv`，并在 `VisitStmt_(AttrStmt)` 里只保留当前核对应的 `resource_scope` 分支（`codegen_npuir_dev.cc:3748-3755`、`codegen_npuir.cc:1315-1323`）。
本次失败日志里的 IR 正好是：
`@FlashAttnExp_mix_aic`（2 个 mmadL1 + `sync_block_set[<CUBE>...]`）和 `@FlashAttnExp_mix_aiv`（vexp + `sync_block_wait[<VECTOR>...]`），外层 `module_core_type = MIX`，两个函数都带 `hivm.part_of_mix, mix_mode = "mix"`。

2. **Developer 模式：发一个 MIX 函数，交给 HIVM 拆。**
不写 `T.Scope` 时，codegen 扫算子：`npuir_dot` / `load_nd2nz` / `store_fixpipe` → hasCube；
其它 `tl.npuir*` → hasVector；两者都有 → MIX（`codegen_npuir_dev.cc:290-310`、`4090-4136`）。
此时 `hasExpert=false`，只发**一个** `func_core_type = MIX` 的函数，真正的 Cube/Vector work-item 拆分由 AscendNPU-IR 的 **CV Pipelining** pass 完成（`cv_pipelining.md`：找循环、把 Cube/Vector 指令拆成独立 Work Item、建立依赖、扩展 buffer）。
模式切换在 `tilelang/engine/lower.py:168-182` expert → `target.build.tilelang_npuir_apis`，developer → `target.build.tilelang_npuir_dev`。
另外 Developer 模式不会传 `--disable-hivm-tensor-compile=true`（`tilelang/jit/jit_npu.py:1486-1495`）。

注意：`T.rs` 的名字是"资源特化"，它不一定触发 MIX；真正触发 expert 拆分的是`T.Scope("Vector")` 产生的 `resource_scope=1`。源码注释写"iff exists T.rs"，但 `T.rs("PIPE_FIX")` 的 attr key 是 `"PIPE_FIX"` 而不是 `"resource_scope"`（`src/ir.cc:366-372`），所以这句注释不精确——**以 IR 为准**。

3.3 1:2 配比

- 硬件：`Ascend910B3.ini:17-33` 明写 `ai_core_cnt=20`、`cube_core_cnt=20`、vector_core_cnt=40、cube_vector_combine=split`。即 **1 个 Cube 核配 2 个 Vector 核**。
- 运行时：1D `T.Kernel(g)` 返回 `(cid, vid)`，`cid` 绑 `blockIdx.x`（逻辑/Cube 块），`vid` 绑 `blockIdx.z`，范围是 `g*2`（`src/ir.cc:178-190`）——`vid` 就是 AIV 子块号 0/1。
- 用法：
  - expert FA：`block_m_half=(block_m+1)//2`，`bx = cid*block_m + subid*real_m`，`real_m = real_m - (tail_size_m % 2)*subid`（L54、L188-190）。UB 里的 `logsum` / `scores_max` / `acc_o` 全是 `block_m_half` 行，两个 AIV 各算一半 M 行。
  - `examples/mixcv_mixkernel.py:65-71` 的注释更直白："two vector cores consume (block_M, block_N) together"，
    `subblock_M = subid * (block_M // 2)`。
- 为什么不是 1:1：一个 Cube 块算完 `block_M×block_N` 的 GEMM 后，softmax 是逐元素/行规约，天然可以按行切给两个 AIV；20 个 Cube 正好喂 40 个 Vector，流水线两端都不闲。
- 代价：M 必须是偶数（`examples/mixcv_mixkernel.py` 里明写 `block_M % 2 == 0`）；expert FA 用 `(tail%2)*subid` 处理奇数尾块，但 `block_m_half` 仍按 `(block_m+1)//2` 分配，最后一个 AIV 可能多算一行空数据。

3.4 Double buffer（双缓冲 / ping-pong）

- 定义：把循环里的一份 buffer 扩成 N 份，第 i 次迭代用第 `i % N` 份；N=2 就是 double buffer。单缓冲时第 i+1 次搬入必须等第 i 次算完（WAR 依赖），三段时间相加；双缓冲进入稳态后耗时 ≈ `max(T_load, T_compute, T_store)`。
- 在 TileLang 里怎么开：

    - Developer 例子用 `T.Pipelined(T.ceildiv(seq_len, block_N), num_stages=2)`（`examples/flash_attn_npuir_dev.py:57`）。`num_stages=2` 让编译器把循环体拆成 2 级软件流水，并为跨级传递的 buffer 做版本化（`src/transform/pipeline_planning.cc`、`multi_version_buffer_rewriter.cc`）。
    - HIVM 侧由 `--enable-auto-multi-buffer=true` 控制，TileLang JIT **默认就传**（`tilelang/jit/jit_npu.py:1465-1471`）。它按内存层次标记候选：GM workspace（`store`/`fixpipe`）、L1（`nd2nz`）、L0C（`fixpipe`）、UB（`load`/`store`）；文档中片上 buffer 固定 N=2，GM workspace 的 N 由 `--set-workspace-multibuffer` 决定（RegBase 默认 2，`multi_buffer.md`、`cv_pipelining.md`）。这些是配置和设计语义，不代表本轮已确认后端最终生效。
    - **GM workspace 的多缓冲是 MixCV 跨核并行的前提**：Cube 的结果经 workspace 交给 Vector，两份 workspace 才能让 Cube 写第 i+1 块时 Vector 还在读第 i 块；只有一份就会互相等。
- 与 CV Pipelining 的关系：CV Pipelining 的阶段数 = workspace 多缓冲的数量；FA 这类融合场景用 Preload/Skew 模式，把 Cube/Vector 包成带 `preload_num` 的 Work Item，通过迭代偏移错开执行（`cv_pipelining.md` 的 Preload 变换示例）。
- 代价：N 倍片上内存；UB/L1/L0C 放不下时编译器会回退（先关 code-motion，再关多缓冲；`--enable-tuning-mode` 可让溢出直接报错）；tile 被迫变小反而更慢；循环次数太少时 prologue/epilogue 摊不薄。
- 本轮的边界：我们只验证了 Developer kernel 能跑、`num_stages=2` 在源码里；**没有做"开/关多缓冲"的对照实验，也没有在 IR 里确认 `hivm.multi_buffer` 标记**（标记在 `bishengir-compile` 内部添加，TileLang 层的 `mlir_content` 看不到）。

3.5 口述版

FlashAttention 在 Ascend 上是 Cube + Vector 两条流水。Cube 只算两个矩阵乘：QK^T 得到 S，PV 得到加权输出；Vector 只做 softmax 和累加：cast、乘 scale、行 max/exp/sum、维护 running max 和 logsum。中间结果放 GM workspace，两条流水用 `sync_block_set/wait` 的 flag 交接。

TileLang 有两种拆法。Expert 模式我写 `T.Scope("Cube")` 和 `T.Scope("Vector")`，codegen 把同一个 prim_func 拆成 `_mix_aic` 和 `_mix_aiv` 两个函数，各带 `hivm.part_of_mix`。Developer 模式我不写 Scope，codegen 看到 `T.gemm`和 v 前缀算子就标成 MIX，只发一个函数，由 AscendNPU-IR 的 CV Pipelining pass自动拆 work item。

910B3 是 20 个 Cube、40 个 Vector，1:2。所以一个 Cube 块配两个 AIV 子块：`T.Kernel` 的第二个返回值 `vid` 范围是块数乘 2，代码里用 `subid` 把 block_M 行一分为二，两个 AIV 各做一半 softmax。

双缓冲由 `T.Pipelined(num_stages=2)` 和 HIVM auto multi-buffer 两层配合。按文档设计，buffer 多槽后可让 Cube 写第 i+1 块时 Vector 仍在读第 i 块；但这次只确认了参数已传，没有做开关 A/B，也没在后端 IR 中确认实际多槽数。